

Joypad Harness: Runtime Safety System for AI Game-Playing Agents

Objective: Build a reusable harness that safely governs any AI game-playing agent by enforcing constraints, measuring progress, handling game state, and escalating failures.

Architecture Flow

Game Frame → Material Handler → Worker Agent → Guardrails → Checkpoints → Alarms → Controller Output

Pillar	Responsibility
Material Handling	Capture screenshots, normalize state, persist checkpoints, replay runs
Guardrails	Prevent impossible inputs, rate-limit actions, enforce controller rules
Checkpoints	Measure progress, confidence, score changes, movement
Alarms	Structured failures with severity, context, and recovery actions

Worker Independence

Workers implement a common interface (decide(state) → controller action). GPT, Claude, local models, or scripted bots can be swapped without changing the harness.

Human Escalation Path

Repeated checkpoint failures trigger a STOP state and request human intervention rather than allowing the agent to continue guessing.

Checkpoint Example

FORWARD_PROGRESS: Mario X-position increased by at least 5 pixels over 60 frames. Pass/Fail result is persisted.

Alarm Example

AGENT_STUCK (HIGH): No forward progress for 120 frames. Recommended Action: switch recovery worker.

Planned Tech Stack

Python • FastAPI • OpenAI/Anthropic-compatible worker interface • SQLite checkpoint storage • Playwright/game capture adapter • JSON alarm pipeline

Demo

Agent attempts gameplay, harness evaluates progress, checkpoint failures generate alarms, and the harness either provides corrective feedback or swaps to a recovery worker. The harness—not the agent—is the primary system being evaluated.